CONFIGURATION REFERENCE

File-by-file explanation and complete configuration snapshots

This reference reflects the project files included in the companion source archive.

1. Flask application

Routes: `/` returns the welcome page; `/health` returns JSON health metadata.

```
from flask import Flask, jsonify
import socket
from datetime import datetime, timezone

app = Flask(__name__)


@app.route("/")
def home():
    return """
    <html>
        <head>
            <title>Docker Compose Project</title>
        </head>
        <body>
            <h1>Welcome to My Docker Compose Project!</h1>
            <p>Flask application running successfully.</p>
            <p>Deployed on Ubuntu using Docker Compose and Nginx.</p>
        </body>
    </html>
    """


@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "hostname": socket.gethostname(),
        "timestamp": datetime.now(timezone.utc).isoformat()
    })


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

2. Python dependencies

Flask is the web framework. Gunicorn is the production-style WSGI process used inside the container.

```
Flask==3.1.2
gunicorn==23.0.0
```

3. Dockerfile

Builds a Python 3.12 slim image, installs pinned dependencies, copies the app, and runs Gunicorn as a non-root user.

```
FROM python:3.12-slim

# Python environment settings
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Set working directory
WORKDIR /app

# Install Python dependencies
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Create a non-root application user
RUN groupadd -r appuser && useradd -r -g appuser appuser

# Copy application files
COPY --chown=appuser:appuser app/ .

# Use non-root user
USER appuser

# Application port
EXPOSE 5000

# Start application using Gunicorn
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

4. Docker Compose

Defines the Flask and Nginx services and joins them to one bridge network.

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: flask_app
    restart: unless-stopped
    expose:
      - "5000"
    networks:
      - app-network

  nginx:
    image: nginx:1.28-alpine
    container_name: nginx_proxy
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - web
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

5. Nginx server block

Listens on port 80, proxies app requests to the `web` service and exposes a direct Nginx health path.

```
server {
    listen 80;
    server_name _;

    client_max_body_size 10M;

    location / {
        proxy_pass http://web:5000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /nginx-health {
        access_log off;
        default_type text/plain;
        return 200 "Nginx is healthy\n";
    }
}
```

6. Git ignore rules

Excludes local Python artifacts, environment files, private key files, and editor/OS files from Git.

```
# Python
__pycache__/
*.py[cod]
*.pyo
.venv/
venv/

# Environment and secrets
.env
.env.*
!.env.example
*.pem
*.key

# OS and editor files
.DS_Store
Thumbs.db
.vscode/
.idea/
```

7. Docker ignore rules

Keeps Git metadata, environment files, docs, assets, screenshots, and local Python artifacts out of the Docker build context.

```
.git
.gitignore
__pycache__/
*.py[cod]
.venv/
venv/
.env
.env.*
screenshots/
docs/
assets/
```

8. Port and network summary

Setting	Configured value	Meaning
Published web port	80:80	Ubuntu host TCP 80 forwards to Nginx container TCP 80.
Flask port	5000	Internal application port. Not directly published on the host.
Compose network	app-network	User-defined Docker bridge network.
Nginx upstream	web:5000	Service name `web`, resolved by Docker Compose DNS.
Nginx config mount	./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro	Read-only bind mount.

9. Operational checks

```
sudo docker compose config
sudo docker compose run --rm nginx nginx -t
sudo docker compose ps
sudo docker compose logs --tail=50 nginx web
curl -i http://localhost/
curl -i http://localhost/health
curl -i http://localhost/nginx-health
```